

ECE 1387 – CAD for Digital Circuit Synthesis and Layout

Exercise #2 – Standard Cell and FPGA Technology Mapping with the ABC Logic Synthesis Framework

Fall 2025

J. Anderson

Assignment date: November 30, 2025
Due date: December 15, 2025, by email to Jason
Late penalty: -1 per day late, with total marks available = 10

The purpose of this exercise is to gain familiarity with the ABC logic synthesis framework, and in particular, use ABC to perform FPGA technology mapping to standard cells and to look-up-tables (LUTs). ABC is a state-of-the-art logic synthesis tool, developed primarily by Alan Mishchenko at UC Berkeley over the past few years¹. As mentioned in class, in ABC, the subject graph consists of 2-input AND gates and inverters – an *AND-inverter graph* (AIG); these are the *base functions* in ABC.

Download ABC from the course website and compile it by typing ‘make’. I verified this works on ECF.

For steps 1-7 of this exercise, you should use the **5** test circuits on the course website. These circuits are from the MCNC and EPFL *benchmark suites* and are commonly used in CAD and architecture research. I suggest you write your own scripts to automate the steps of this exercise for the 5 circuits. The test circuits are in BLIF (Berkeley Logic Interchange Format). Take a look at one of the BLIF inputs: the circuit is represented as a directed acyclic graph, where the nodes represent logic functions. BLIF gives each node’s logic function as a truth table. Don’t-cares in BLIF truth tables are represented using a hyphen(-).

For technology mapping, you will perform both standard cell and FPGA mapping for the test circuits. Both styles of mapping use an algorithm is very similar to the DAGON algorithm used in class. Namely, it applies dynamic programming to map the AIG circuit into standard cells or LUTs.

What to do and what to hand in?

1. FPGA mapping: Start ABC by typing `abc`. Read each circuit into ABC, perform technology independent optimization (akin to Karnaugh map optimization), and map the circuit to 6-input LUTs. ABC commands for the circuit `alu4.blif`:

```
read_blif alu4.blif
resyn2 // technology indep. optimization (you need to have abc.rc in your directory)
print_stats // prints details about the AIG
if -K 6 // technology mapping to 6-input LUTs
print_stats // prints details about the mapping solution
write_blif alu4.mapped.blif // writes the mapping in BLIF format
```

¹ <http://www.eecs.berkeley.edu/~alanmi/abc/>

The output from the `print_stats` command before mapping gives the number of 2-input AND gates in the AIG; the number of inverters is not reported as inversion is managed as an attribute on edges of the AIG. The output from the `print_stats` command after mapping gives the number of LUTs (`nd`) in the mapping solution and the number of LUT levels on the critical path (`lev`) – i.e., the mapped circuit depth. For each test circuit, report the number of LUTs in the mapping, and its depth. Take a look at the mapped BLIF and you will see that each node uses at most 6-inputs, and can therefore be implemented in a 6-input LUT.

2. By default, the priority cuts mapping algorithm in ABC optimizes circuit depth. With the `-a` option, the mapper can optimize silicon area (# of LUTs). Repeat step #1 above (6-LUT mapping), optimizing for area (if `-K 6 -a`). By how much was area reduced, on average, versus the results in step #2? How much was depth affected?
3. Technology-independent optimization is a bit of “black magic”. In some recent work, we noticed that calling `resyn2` multiple times can impact the implementation somewhat. Do step #1 again, calling `resyn2` three times instead of once. Comment on how that impacts the results.
4. 6-LUTs in recent Xilinx FPGAs are “fracturable”. Such LUTs can implement any 6-input function, or any *two* functions that *together* use no more than 5 unique input signals. Using the output of step #2 above as input, write a program (in your preferred language) to pack the LUTs into a minimum number of fracturable Xilinx-like 6-LUTs. The output of your program should be in the form of LUT output signal (net) names on lines like this:

```
net1name
net2name net3name
```

where the first line indicates a single LUT with output signal name `net1name` is packed into a fracturable LUT, and the second line indicates two LUTs are packed into a single fracturable LUT, with their output signal names being `net2name` and `net3name`.

Describe how your program works and the algorithm used for packing LUTs. Be innovative to minimize the number of fracturable LUTs used in the mapping. Report the # of fracturable LUTs used for each test circuit. Include your program source code in the email to Jason.

5. Standard-cell mapping: Now, perform mapping to standard cells for the circuits. Use the following commands:

```
read_blif alu4.blif
resyn2 // tech. independent optimization
read_library mcnc.genlib // use mcnc.genlib from course website
map -v // TM to standard cells
print_stats
print_gates
```

```
write_blif alu4.mapped.sc.blif
```

The output from `print_stats` gives the area and delay of the standard-cell mapping, as well as the # of cell levels on the critical path. Report these three items for each circuit.

6. Create a copy of `mcnc.genlib` and open the copy. You will see the standard-cell gates, their logic functions, area, and delay information. Delete all of the gates except for `inv1`, `nand2`, `buf`, `zero`, and `one`. Thus, you are creating a “minimal” gate library. Repeat step #5. If you wish to know about the library format: https://workcraft.org/_media/help/sis_genlib.pdf
7. An interesting question to investigate is the following: How many NOR gates is a 6-LUT equivalent to? For each circuit, compute the number of NOR-gate-equivalents used for the mappings produced by step #6 as follows: $\text{NAND-gate-equivalents} = \#nand2 + 0.5 * \#inv1$. Compare this quantity, for each circuit, with the LUT counts from steps 1 and 2. Present the LUT counts (from steps #1 and #2) and NAND-gate-equivalent results in a table, with a row for each circuit. On average, for each circuit, how many NAND-gate-equivalents does each LUT implement?